

Arquitectura del Hardware

Autor: Diego Navarro Sánchez

Laboratorio de Elementos Programables I — Sesión 02: Reto 02

1. ¿Qué hay dentro del microcontrolador?

Un microcontrolador integra CPU, memoria y periféricos en un solo chip de silicio. A diferencia de una PC, no ejecuta un sistema operativo tipo Linux ni múltiples procesos, y su memoria se mide en kilobytes en lugar de gigabytes. La ingeniería embebida consiste en hacer mucho con muy poca memoria, y en saber exactamente dónde vive cada dato.

2. Mapa de Memoria (RP2040 / RP2350)

- **Stack:** Crece hacia abajo (0x20041FF4). Almacena variables locales como `stack_value`.
- **Heap:** Crece hacia arriba (0x20001198). Memoria dinámica reservada con `malloc()`.
- **Variables Globales:** Ubicadas en la SRAM base (0x20001040).
- **Flash / XIP:** Almacena el programa y constantes (0x10004A2C).

Direcciones clave: Flash vive en 0x10000000. SRAM vive en 0x20000000. Los periféricos ocupan los rangos altos 0x40000000 (APB) y 0xD0000000 (SIO, GPIO rápido).

3. Registros: El puente al mundo físico

Un registro es una dirección de memoria mapeada directamente al hardware interno, no a una variable normal en RAM. Escribir en un registro altera físicamente el estado de los pines, temporizadores o interrupciones.

```
sio_hw->gpio_set = 1u << LED_PIN;
```

- **BIT = 0 (LOW):** Salida a $\approx 0V$.
- **BIT = 1 (HIGH):** Salida a $\approx 3.3V$.

CÓDIGO → CPU → BUS → REGISTRO → GPIO (3.3V)

4. Del Programa al Silicio: MicroPython vs C/C++

MicroPython	C / C++ (Pico SDK)
• Traduce el código línea por línea en tiempo de ejecución. • Utiliza el módulo machine como abstracción de hardware. • Ideal para prototipado rápido, aunque añade overhead de memoria.	• Se compila directamente a binario/código máquina antes de ejecutarse. • Manipula registros mediante funciones nativas del SDK (gpio_init, malloc). • Permite consultar direcciones de memoria reales con printf("%p", ...).

5. Comparativa de Hardware: RP2040 vs RP2350

Característica	RP2040 (Pico W)	RP2350 (Pico 2 W)
CPU	2x Arm Cortex-M0+	2x Cortex-M33 / 2x Hazard3 RISC-V
SRAM	264 KB	520 KB
PIO (State Machines)	8	12
Board Target (SDK)	pico_w	pico2_w
Enfoque / Rol	Chip simulado en Wokwi	Placa objetivo del curso

6. Evidencia de ejecución: Wokwi

Evidencia C/C++ (Pico SDK)	Evidencia MicroPython
<pre>flash const: 10004A2C global: 20001040 stack value: 20041FF4 heap ptr: 20001198</pre> Demuestra la separación física entre la memoria Flash y las distintas regiones dentro de la RAM (SRAM).	<pre>Frecuencia CPU (Hz): 125000000 libre inicial: 240912 después de reservar: 230896 después de liberar: 240912</pre> Reserva de 10,000 bytes. El recolector de basura (gc.collect()) recupera totalmente la memoria libre.

7. Alcance y Referencias

- **Nota de simulación:** No se utilizó hardware físico en esta entrega. Toda la evidencia fue generada en el simulador Wokwi, que emula el chip RP2040 y no el RP2350 de la RASPBERRY PI PICO 2W.
- **Referencias:**
 - Raspberry Pi Pico C/C++ SDK Documentation.
 - Raspberry Pi Pico MicroPython Documentation.
 - Wokwi Raspberry Pi Pico Documentation.